

BCD I: Business Components Development I

Foundations of Java EE and Distributed Systems

Prepared by: K A A C Lakshan

Senior Lecturer

Java Institute for Advanced Technology (JIAT)

Contents

1. J2EE Introduction & Architecture	3
1.1 Multi-Tier Architecture	3
Diagram: J2EE Multi-Tier Architecture	3
2. Cloud Computing Paradigms	5
2.1 Service Models	5
2.2 Java EE in the Cloud	5
3. CORBA (Common Object Request Broker Architecture)	6
3.1 Core Components	6
4. Java RMI (Remote Method Invocation)	7
4.1 RMI Architecture	7
Diagram: RMI Communication Flow	7
5. Enterprise Beans (EJB) Fundamentals	8
5.1 Types of Session Beans	8
6. Structure of a WAR file	9
6.1 WAR Directory Layout	9
7. Structure of an EJB Jar	10
7.1 EJB Jar Layout	10
8. Structure of an EAR file	10
8.1 EAR Directory Layout	10
Diagram: EAR Classloading Hierarchy	11
9. Portable JNDI Syntax	12
9.1 The Three JNDI Namespaces	12
10. Java Message Service (JMS)	13
10.1 Messaging Domains	13
10.2 JMS Components	13
Diagram: JMS Queue vs Topic	14

1. J2EE Introduction & Architecture

Java 2 Platform, Enterprise Edition (J2EE), now formally known as Jakarta EE, is a set of specifications extending Java SE with specifications for enterprise features such as distributed computing, web services, and component models. BCD I sets the foundation for these technologies.

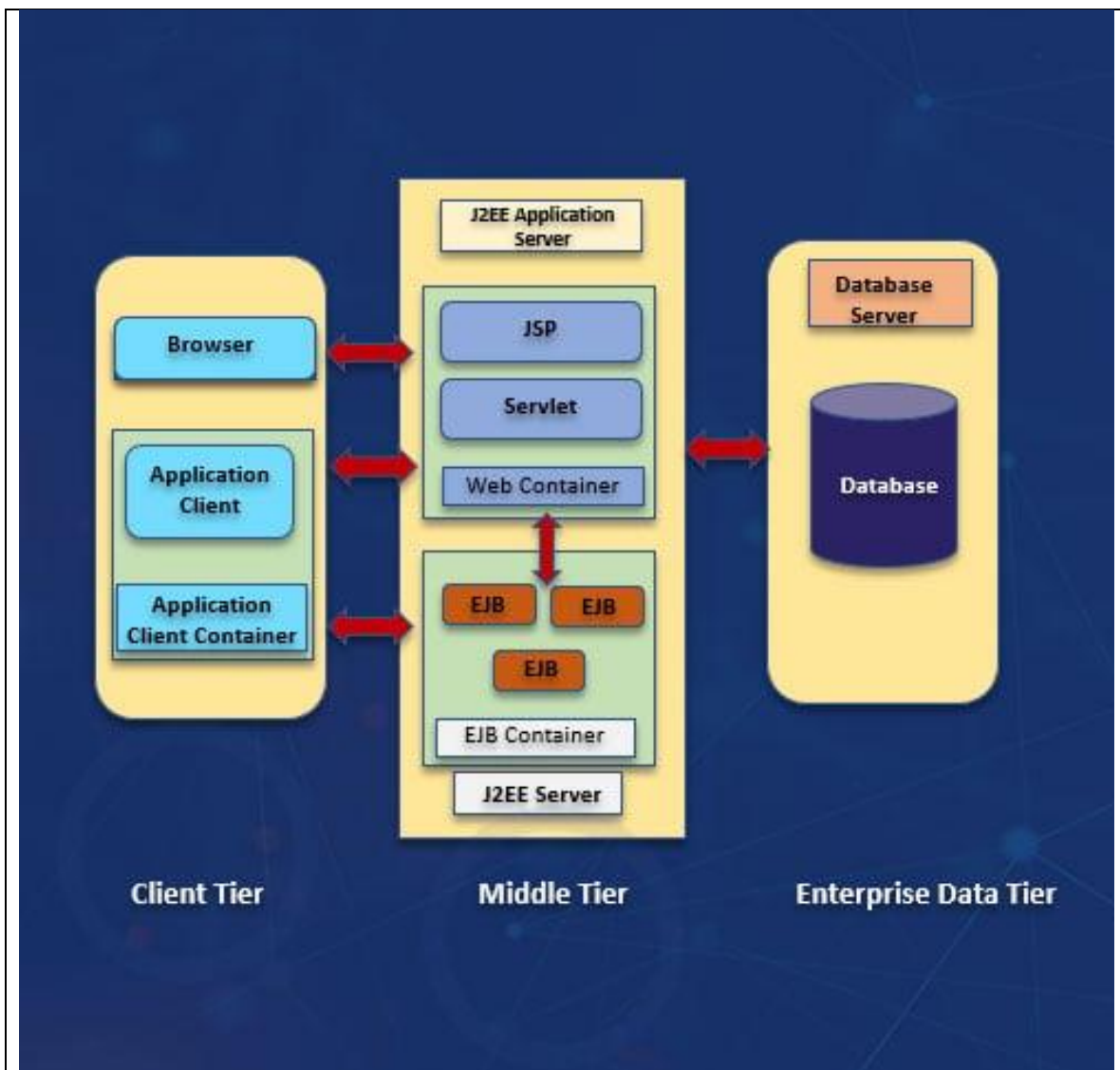
1.1 Multi-Tier Architecture

J2EE defines a multi-tiered distributed application model. Application logic is divided into components according to function, and the various application components that make up a J2EE application are installed on different machines depending on the tier in the multi-tiered J2EE environment to which the application component belongs.

- Client Tier: Browsers, Java Applets, or standalone applications.
- Web Tier: Servlets, JSP pages, and JSF components handling HTTP requests.
- Business Tier: Enterprise JavaBeans (EJB) containing core business logic.
- Enterprise Information System (EIS) Tier: Databases, mainframe transaction processing, legacy systems.

Diagram: J2EE Multi-Tier Architecture

Description: A diagram showing four distinct vertical layers: Client Tier (Browsers/Mobile), Web Tier (Web Server/Container), Business Tier (EJB Container), and EIS Tier (Database). Arrows show the bidirectional flow of data across tiers.



[J2EE Multi-Tier Architecture]

2. Cloud Computing Paradigms

Cloud computing is the on-demand availability of computer system resources, especially data storage (cloud storage) and computing power, without direct active management by the user. Modern Java EE applications are often deployed to cloud environments.

2.1 Service Models

- Infrastructure as a Service (IaaS): Provides raw compute, network, and storage. (e.g., AWS EC2).
- Platform as a Service (PaaS): Provides a managed platform to deploy applications without managing the underlying OS. (e.g., Heroku, AWS Elastic Beanstalk).
- Software as a Service (SaaS): Fully managed software applications accessible via a browser. (e.g., Google Workspace, Salesforce).

2.2 Java EE in the Cloud

Deploying Java components in the cloud involves containerization (like Docker) and orchestration (like Kubernetes). A microservices architecture, heavily influenced by cloud design patterns, breaks down large monolithic EAR files into smaller, independently deployable services.

3. CORBA (Common Object Request Broker Architecture)

CORBA is an early standard for distributed computing designed to facilitate the communication of systems that are deployed on diverse platforms. It enables collaboration between systems on different operating systems, programming languages, and computing hardware.

3.1 Core Components

- ORB (Object Request Broker): The middleware that establishes the client-server relationships between objects.
- IDL (Interface Definition Language): A declarative language used to define the interfaces of the distributed objects. IDL is language-neutral and is compiled into client stubs and server skeletons.

```
// Example CORBA IDL
module Banking {
    interface Account {
        void deposit(in double amount);
        double getBalance();
    };
};
```

Best Practice: While CORBA is largely considered a legacy technology in modern development (superseded by REST, gRPC, and SOAP), understanding its ORB mechanism is crucial as it formed the underlying foundation for RMI-IIOP in early Java EE versions.

4. Java RMI (Remote Method Invocation)

Java RMI is a Java API that performs remote procedure calls (RPC). It allows an object residing in one Java Virtual Machine (JVM) to invoke methods on an object running in another JVM. RMI is native to Java and simplifies distributed object communication.

4.1 RMI Architecture

RMI relies on the concept of Stubs and Skeletons (historically) or dynamic proxies in modern Java. The client interacts with a Stub acting as a proxy. The request is serialized, sent over the network, and the Skeleton (or remote dispatcher) unpacks it, invokes the actual method, and returns the serialized result.

```
// 1. Remote Interface
import java.rmi.Remote;
import java.rmi.RemoteException;

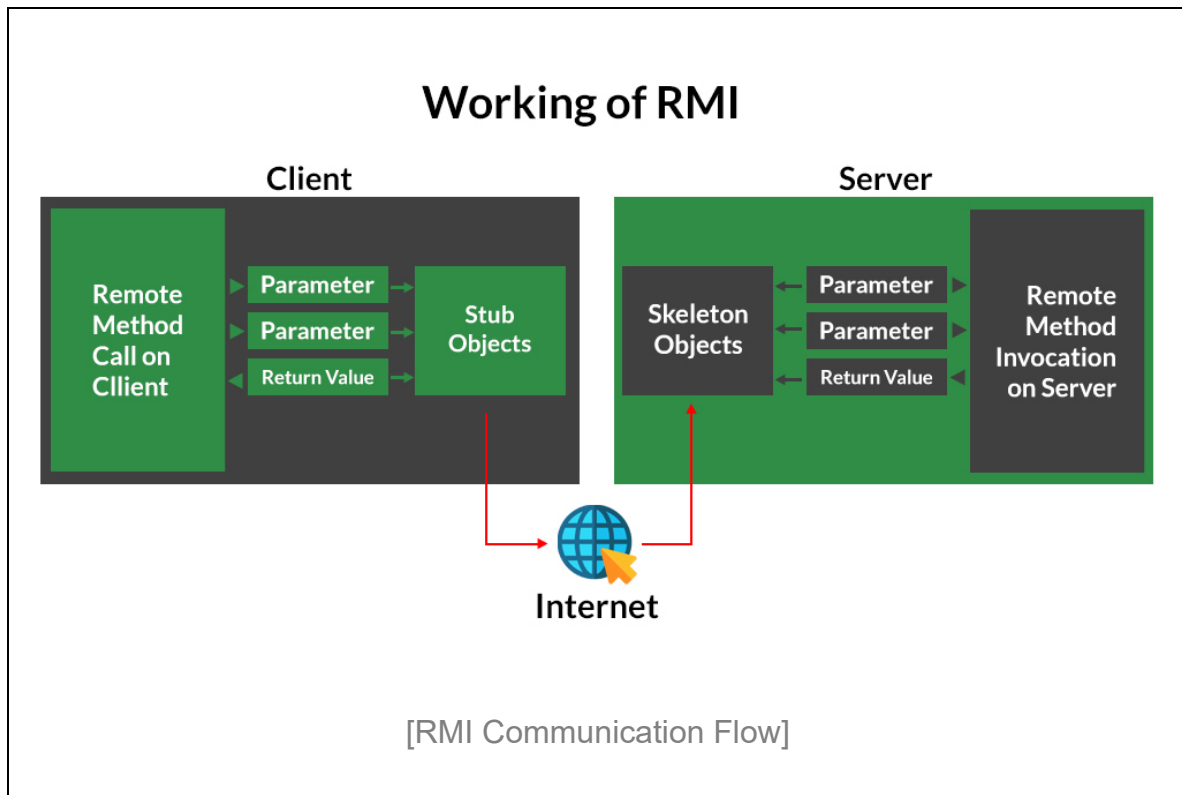
public interface Calculator extends Remote {
    int add(int a, int b) throws RemoteException;
}

// 2. Server Implementation
import java.rmi.server.UnicastRemoteObject;

public class CalculatorImpl extends UnicastRemoteObject
implements Calculator {
    protected CalculatorImpl() throws RemoteException {
        super();
    }
    public int add(int a, int b) { return a + b; }
}
```

Diagram: RMI Communication Flow

Description: Two JVMs side-by-side. Client JVM has the Client Object and Stub. Server JVM has the Skeleton and Remote Object. The network layer separates them. Arrows show method call to Stub, network transmission, Skeleton receiving, calling Remote Object, and returning the result.



5. Enterprise Beans (EJB) Fundamentals

Enterprise JavaBeans (EJB) is the server-side component architecture for Java EE. EJB technology enables rapid and simplified development of distributed, transactional, secure, and portable applications based on Java technology.

5.1 Types of Session Beans

- Stateless Session Beans (@Stateless): Do not maintain a conversational state with the client. When a client invokes a method, the bean's state may change for the duration of the invocation, but is not retained. They are highly scalable.
- Stateful Session Beans (@Stateful): Maintain a conversational state across multiple method invocations for a single client (e.g., a shopping cart).
- Singleton Session Beans (@Singleton): Instantiated once per application and exist for the life of the application. Used for sharing state globally or caching.


```

@Stateless
public class InventoryBean implements InventoryRemote {
    public void updateStock(String item, int quantity) {
        // Business logic to update inventory
    }
}

```

6. Structure of a WAR file

A Web Archive (WAR) encapsulates web components like Servlets, JSPs, HTML, CSS, JavaScript, and associated Java classes into a single deployable unit.

6.1 WAR Directory Layout

The structure is strictly defined by the Java EE specification. Static resources (HTML, CSS) sit at the root. The WEB-INF directory is hidden from direct client access and contains the core application files.

```

myapp.war
|-- index.html
|-- css/
|   |-- styles.css
|-- WEB-INF/
|   |-- web.xml (Deployment Descriptor)
|   |-- classes/ (Compiled Servlet and utility classes)
|       |-- com/
|           |-- jiat/
|               |-- MyServlet.class
|   |-- lib/ (Third-party JAR files)
|       |-- gson.jar

```

7. Structure of an EJB Jar

An EJB JAR is a standard Java Archive (JAR) specifically structured to contain enterprise beans.

7.1 EJB Jar Layout

It contains the compiled .class files for the EJBs and their interfaces, along with a META-INF directory for deployment descriptors. In modern Java EE, the ejb-jar.xml is optional due to annotations, but the structure remains the same.

```
myejb.jar
|-- com/
|   |-- jiat/
|   |   |-- ecom/
|   |       |-- OrderProcessor.class
|   |       |-- OrderProcessorBean.class
|-- META-INF/
|   |-- MANIFEST.MF
|   |-- ejb-jar.xml (Optional deployment descriptor)
```

8. Structure of an EAR file

An Enterprise Archive (EAR) is the top-level deployment unit. It packages one or more Web modules (WARs) and EJB modules (JARs) into a cohesive enterprise application.

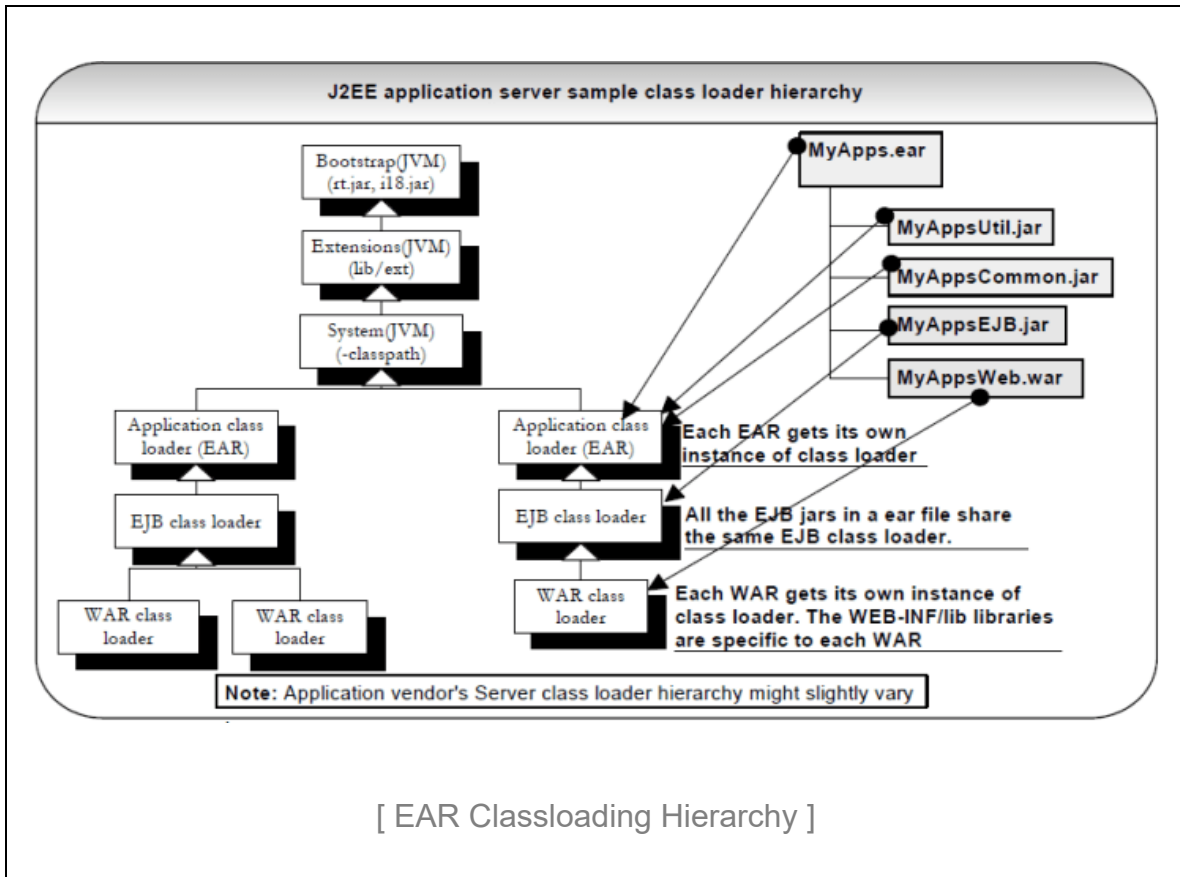
8.1 EAR Directory Layout

The EAR contains a META-INF directory with an application.xml descriptor, which maps the internal modules and assigns a context root to the web applications.

```
myenterpriseapp.ear
|-- myweb.war (Web Module)
|-- myejb.jar (EJB Module)
|-- lib/ (Shared libraries for all modules)
|   |-- shared-utils.jar
|-- META-INF/
|   |-- MANIFEST.MF
|   |-- application.xml (Describes modules and context roots)
```

Diagram: EAR Classloading Hierarchy

Description: A tree showing the Application Server Classloader at the top, delegating to the EAR Classloader (shared libs), which then delegates down to isolated Classloaders for the WAR and EJB JARs.



9. Portable JNDI Syntax

The Java Naming and Directory Interface (JNDI) allows applications to discover and look up data and objects via names. Java EE 6 introduced portable JNDI names to resolve issues with vendor-specific JNDI implementations, making EJB lookups consistent across all application servers.

9.1 The Three JNDI Namespaces

1. `java:global`: Used to look up a component from anywhere on the application server, even from a different deployed EAR.

Format: `java:global[/application-name]/module-name/enterprise-bean-name[/interface-name]`

2. `java:app`: Used to look up a component within the same application (EAR).

Format: `java:app/module-name/enterprise-bean-name[/interface-name]`

3. `java:module`: Used to look up a component within the same module (e.g., inside the same EJB JAR).

Format: `java:module/enterprise-bean-name[/interface-name]`

```
// Example of a Portable JNDI Lookup
Context ctx = new InitialContext();
InventoryRemote inventory = (InventoryRemote)
    ctx.lookup("java:global/ecommerce-ear/ecommerce-
ejb/InventoryBean!com.jiat.InventoryRemote");
```

10. Java Message Service (JMS)

JMS is a Java API that allows applications to create, send, receive, and read messages. It enables loosely coupled, reliable, and asynchronous communication between distributed components.

10.1 Messaging Domains

- Point-to-Point (Queues): A message is sent to a specific queue. Only one consumer will receive and process that message. Ensures exactly-once processing.
- Publish-Subscribe (Topics): A message is sent to a topic. Multiple subscribers who are listening to that topic will all receive a copy of the message. Useful for event broadcasting.

10.2 JMS Components

To send or receive a message, a developer needs a `ConnectionFactory` and a `Destination` (Queue or Topic), both typically obtained via JNDI lookup.

```
// Sending a Message via JMS
@Resource(mappedName="jms/MyConnectionFactory")
private ConnectionFactory connectionFactory;

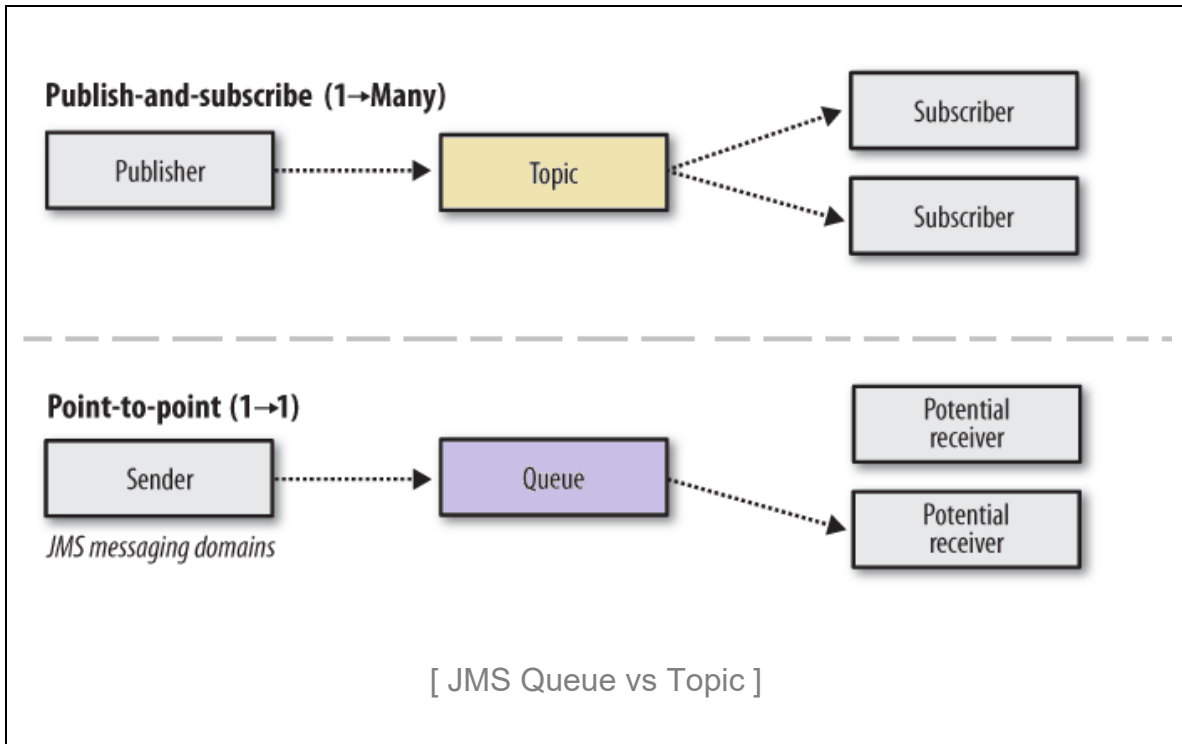
@Resource(mappedName="jms/MyQueue")
private Queue queue;

public void sendMessage(String text) {
    try (Connection connection =
        connectionFactory.createConnection();
        Session session = connection.createSession(false,
            Session.AUTO_ACKNOWLEDGE);
        MessageProducer producer =
            session.createProducer(queue)) {

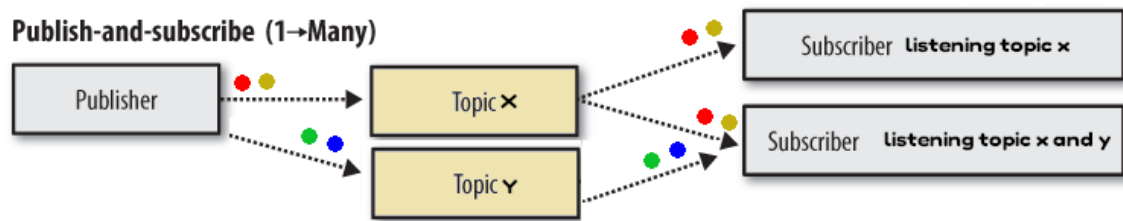
        TextMessage message = session.createTextMessage(text);
        producer.send(message);
    } catch (JMSEException e) {
        e.printStackTrace();
    }
}
```

Diagram: JMS Queue vs Topic

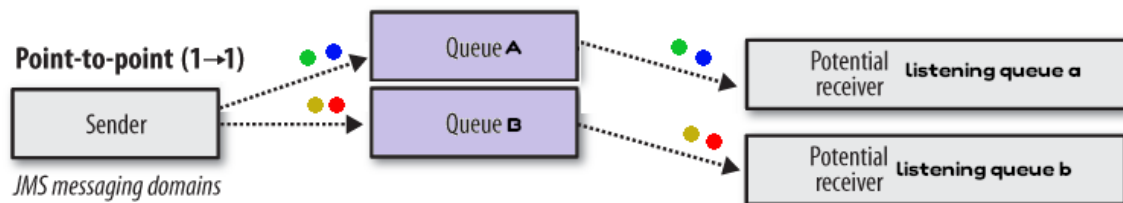
Description: A split diagram. Top half shows Point-to-Point: Sender -> Queue -> Single Receiver. Bottom half shows Publish-Subscribe: Publisher -> Topic -> Multiple Subscribers simultaneously receiving the message.



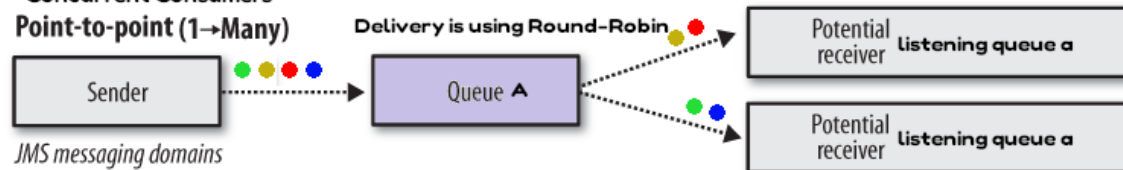
Publish-and-subscribe (1→Many)



Point-to-point (1→1)



Concurrent Consumers Point-to-point (1→Many)



[JMS Queue vs Topic]